

Vector Search on MongoDB Atlas

LUMINA gold corpus, document 2 of 4. FDE Agent Engineering Bootcamp. CC BY 4.0.

One database, vectors included

MongoDB Atlas Vector Search stores an embedding as an ordinary array field on an ordinary document. A chunk's text, its locator, its owning document and space, and its vector all live in the same record, which means a citation is one document read rather than a join across a vector store and a database that must agree about ids.

The practical consequence is that a filter like "only this Space" is a query predicate rather than a distributed-systems problem, and there is no second store to keep in sync, back up, or explain when the two disagree.

Index definitions

A vector index is declared as a list of fields. The vector field names its path, its dimensionality, and its similarity function; filter fields name paths that queries may constrain.

```
{
  "fields": [
    { "type": "vector", "path": "embedding", "numDimensions": 1536, "similarity": "cosine" },
    { "type": "filter", "path": "spaceId" }
  ]
}
```

numDimensions must match the embedding model exactly. Supported similarity functions are cosine, euclidean, and dotProduct; cosine is the default choice for text embeddings because it ignores vector magnitude, which carries no meaning for a normalized text embedding.

A field must be declared as a filter field at index time to be usable as a filter at query time. This is the single most common misconfiguration in a first RAG build, and its symptom is subtle: the query succeeds and returns the wrong documents.

Filtering inside the search, not after it

`$vectorSearch` returns approximate nearest neighbours. It examines a bounded candidate set and returns the best limit of them. Where the filter is applied changes the result:

" Filter inside `$vectorSearch`. The candidate search itself is constrained, so all limit results come from the right Space.

" Filter in a later \$match. The search returns the globally nearest neighbours, and the \$match then deletes the ones from other Spaces. If the query's neighbours mostly belong to another Space, you get two results where you asked for ten or none.

The second shape does not error. It quietly under-retrieves, recall drops, and the pipeline looks correct in code review. The rule is: the filter belongs inside the search stage.

A related parameter is numCandidates, the size of the candidate pool the approximate search considers before returning limit results. Setting it close to limit makes the search fast and imprecise; the usual guidance is to set numCandidates well above limit on the order of 10 to 20 times and tune down only if latency demands it.

Text search and hybrid queries

Atlas Search is the BM25 half of a hybrid retriever, defined as a separate index with mappings. A static mapping ("dynamic": false) names each field and its type: string for text to be analyzed and scored, token for values to be matched exactly, such as an id used as a filter.

Running a \$search stage and a \$vectorSearch stage and fusing the two result lists with reciprocal rank fusion gives one ranking from both retrievers. The two stages cannot be combined in a single aggregation stage; they are separate pipelines whose results are merged, which is why RRF over ranks is the natural fusion here.

Eventual consistency: upserted is not searchable

Atlas Search indexes, vector and text alike, are eventually consistent. A write to a collection returns as soon as the write is durable. The index that makes the document searchable is updated asynchronously, typically within seconds, and there is no guarantee attached to that interval.

This produces the most confusing bug in a document-ingestion pipeline: the pipeline reports success, the chunks are visibly in the collection, and a query for them returns nothing. Nothing is broken. The index has not caught up.

The fix is a read-your-write probe. After writing chunks, query the vector index for one of the chunks just written and wait until it comes back. Only then mark the document searchable indexed, or whatever the status field is called. A status that means "upserted" while claiming to mean "searchable" is a lie the rest of the system will believe.

listSearchIndexes reports an index's status and a queryable flag. An index that is still building will answer a query, and answer it from an incomplete picture of the data, so a recall number measured during a build is not a measurement of anything.

What the free tier allows

The course's declared constraint for this assignment: a free-tier M0 Atlas cluster allows three search indexes and 512 MB of storage. LUMINA needs exactly three: a vector index on memories, a vector index on chunks, and a text index on chunks. A fourth requires a paid tier, so the budget is spent and the architecture must fit it.

512 MB is generous for text and tight for vectors. A 1536-dimension vector stored as 64-bit floats occupies about 12 KB before overhead, so roughly 40,000 chunks fills the tier. A small document corpus is fine; a corpus of a thousand PDFs is not.

GridFS for the raw files

Embeddings answer queries; the original file still has to live somewhere. GridFS stores a file as a document in a files collection plus binary chunks in a chunks collection, which is enough infrastructure for uploads and generated artifacts without provisioning an object store. It is not a CDN and should not be asked to be one, but for a course project it removes a dependency.

Local development without Atlas

A plain mongod from Docker or a local install has neither Atlas Search nor Atlas Vector Search. `$vectorSearch` is not a MongoDB aggregation stage; it is an Atlas feature.

The documented fallback is a brute-force cosine scan: load a Space's chunks, compute cosine similarity in application code, sort, and take the top k. It is correct, it is exact rather than approximate, and it is linear in the number of chunks. Up to a few thousand chunks it is fast enough to develop against; past that it is unusable.

Whichever backend is live must be named by the service's health endpoint. A grader looking at a recall number needs to know whether it came from an approximate index or an exact scan, because the two are not the same measurement.